Propeller Project

Line Following Robot with Obstacle and Object Detection for Contact-Less Delivery

Group 19

1) **Dajr Alfred**
2) **Gordon Oboh**
3) **Mohammed Nauman Shariff**

04/20/2022

NYU

Contents

1. Introduction

Brief Description of the scenario

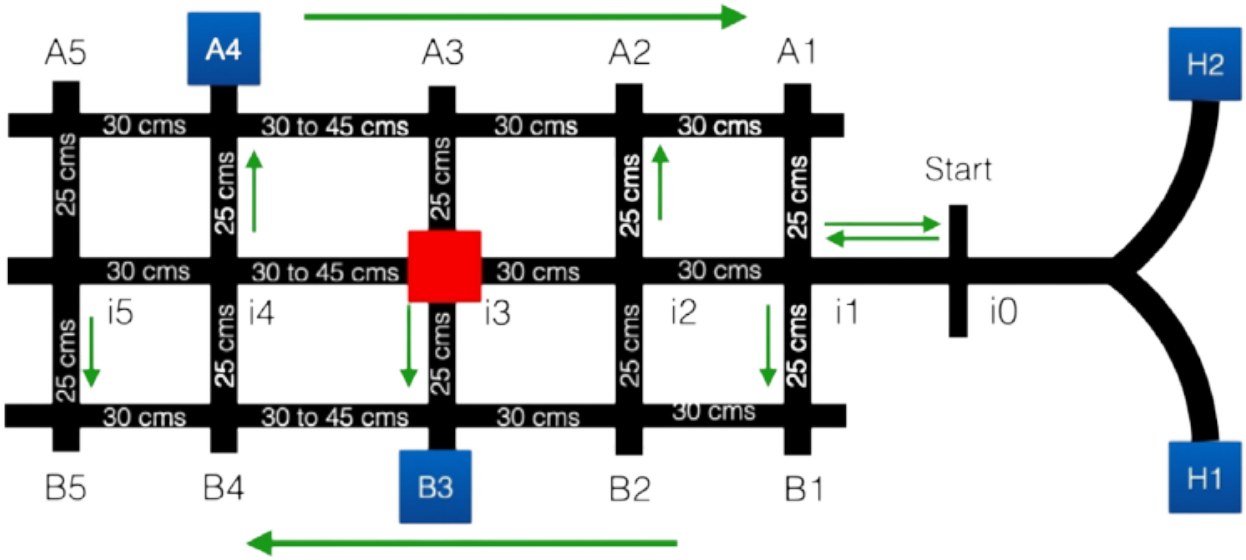

Figure 1.1: The arena, A grid-like structure

1.1 Problem Objective and its Solution

With the emergence of COVID-19 day to day activities have been impeded. Due to this unusual situation, there's a need for a contact less delivery system. The delivery system will make delivery of COVID test kits possible for individuals who are unable to go out to purchase a test kit. Figure 1 shows an outline of the map in a grid-like structure similar to the streets of Manhattan and some parts of Brooklyn. There are two home locations (H1 and H2) from there the robot starts on the track and there are 16 intersections (i0 to i5, B1 to B5, and A1 to A5) with 10 possible delivery points and an obstacle that is placed randomly in either i2, i3 or i5. The green arrows represent the direction of traffic flow on each street and the center lane is bi-directional.

Our implementation of the line following robot uses an eight-channel digital IR Reflectance sensor to follow the line and detect intersections. If the leftmost IR sensor detects black tape the Arduino compensates and tries to put the robot back on track by moving towards the right, if the rightmost IR

sensor detects black tape the arduino compensates for the error by moving towards the left. Using this mechanism, the bot follows the direction of the tape, but rather than stopping to scan, the propeller's multicore architecture allows the robot to perform all these operations simultaneously or at least faster than a single core microcontroller.

2. Methodology

2.1 Circuit Diagram and explanation

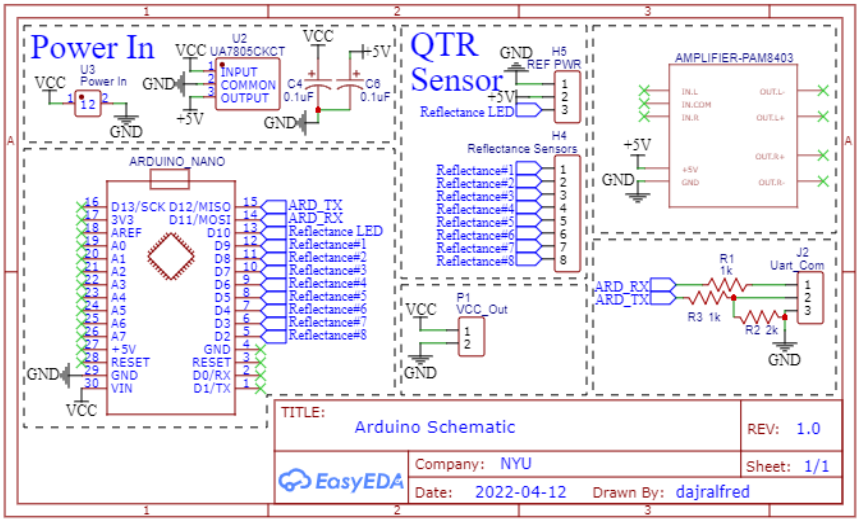

Figure 2.1: Arduino Circuit Diagram

Figure 2.1 shows how the Arduino Nano is connected to the QTR sensor and the Pins that are used to send the data via Serial Communication.

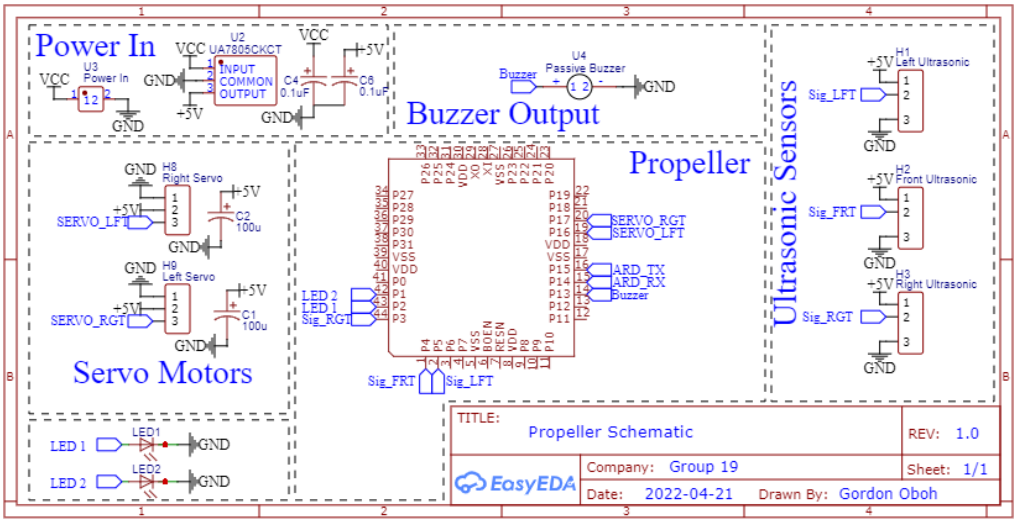

Figure 2.2: Propeller Circuit Diagram

Figure 2.2 shows how the Propeller is connected to the servo motors, LEDs, buzzer, and Ultrasonic sensors and the pins used for Serial communication with the Arduino Nano.

2.2 Pin Representation

Arduino Connection:

- Pin D2 - Reflectance Pin 8.
- Pin D3 - Reflectance Pin 7.
- Pin D4 - Reflectance Pin 6.
- Pin D5 - Reflectance Pin 5.
- Pin D6 - Reflectance Pin 4.
- Pin D7 - Reflectance Pin 3.
- Pin D8 - Reflectance Pin 2.
- Pin D9 - Reflectance Pin 1.
- Pin D10 - Reflectance LED Pin.
- Pin D11 - UART Receiver Pin.
- Pin D12 - UART Transmitter Pin.

Propeller Connection:

- Pin 1 - LED 2.
- Pin 2 - LED 1.
- Pin 3 - Signal Pin for Right Ultrasonic sensor.
- Pin 4 - Signal Pin for Center Ultrasonic sensor.
- Pin 5 - Signal Pin for Left Ultrasonic sensor.
- Pin 13 - Buzzer.
- Pin 14 - UART Transmitter Pin.
- Pin 15 - UART Receiver Pin.
- Pin 16 - Signal Pin for Left Servo Motor.
- Pin 17 - Signal Pin for Right Servo Motor.

An Arduino Nano was used due to space constraints and to act as a slave to a Propeller Master, because of Arduino's open-source nature it has support for the QTR Sensory library, unlike the propeller. Instead of trying to "reinvent the wheel", it was better to use the Arduino and its access to the needed library to read the data from the QTR sensor. Later send that data via a Serial Communication to the Propeller, while the propeller uses this data in its decision making and branching.

3. Software Session

3.1 Propeller Functions and Description

getPos(); This function starts a cog and the cog is used to read the values of the QTR sensor sent by the arduino nano via serial communication to Propeller.

getDist(); This function uses the built-in built command *ping_cm* to read the length of the return pulse and calculate the time in centimeters.

soundBuzzer(); This function starts a cog and is used to trigger the buzzer and then the cog is stopped at the end.

lineFollower(); This function starts a cog and is used to ensure the robot follows the line, the function compares the difference of the present position from the desired position to a defined error threshold, and then the cog is stopped at the end.

detectIntersection(); This function compares the 8 values from the QTR sensor to a reference threshold and returns a 0 or 1.

intersectionSequence(); This function triggers the buzzer.

Follow(); This function is used to start the cog for *lineFollower()*.

spinServos(); This function uses a switch statement as a conditional to select the speed and direction of the servo, below is a brief explanation of the result of each case.

- **Case 0**; this case uses the *servo_set* command to stop the servos.
- **Case 1;** this case uses the *servo_speed* command to make the servos move the robot forward.
- **Case 2**; this case uses the *servo_speed* command to make the servos spin the robot left on its wheel axis.
- **Case 3**; this case uses the *servo_speed* command to make the servos spin the robot right on its wheel axis.
- **Default Case**; this case sets the switch expression to case 0.

This function runs on an independent cog, uses a while loop to make it repeat, and is also the backbone for the functions below.

Forward(); it takes one parameter for the speed and passes that to *spinServos()* with the switch expression *motorDir* set to 1.

TurnLeft(); it takes one parameter for the speed and passes that to *spinServos()* with the switch expression *motorDir* set to 2.

TurnRight(); it takes one parameter for the speed and passes that to *spinServos()* with the switch expression *motorDir* set to 3.

Stop(); it takes one parameter for the speed and passes that to *spinServos()* with the switch expression *motorDir* set to 0.

Turn180(); it takes no parameter and uses the switch expression *motorDir*(set to 3) with *pause()* command to make the robot spin 180° on its wheel axis.

Turn90L(); it takes no parameters and uses the switch expression *motorDir*(set to 3) with *pause()* command to make the robot turn left on another lane.

Turn90R(); it takes no parameters and uses the switch expression *motorDir*(set to 2) with *pause()* command to make the robot turn right on another lane.

3.2 Arduino Code

The Arduino code is used to read the QTR sensor and send the data to the Propeller via Serial communication. The code initializes 11 pins, Pin 1 to 8 for each array on the QTR sensor, Pin 10 for the emitter control pin, Pin 11 and 12 for Receiving and Transmitting data respectively. In the setup part, the QTR sensor is set to a digital mode and there's a calibration sequence to allow the QTR sensors to be calibrated for each run. The Loop section is used to keep reading the values from each sensor pin.

3.3 Propeller Code

The Propeller uses a switch statement to control the flow of the program and trigger an appropriate response based on inputs and conditions, below is a brief explanation of the result of each case;

- **Case 0**; this case gets QTR Sensor data from the Arduino and uses that to follow the line from H1 or H2 to intersection i0.
- **Case 1**; this case triggers the front ultrasonic and it's used to scan for obstacles in front of the robot, once the robot moves beyond intersection i0.
- **Case 2**; the front ultrasonic is used to approach an obstacle, then this case is activated and it returns the robot to a i1 and then it branches off to the B1.
- **Case 3 and 4**; these case statements use the left ultrasonic sensor to scan the delivery locations for objects and an object counter is incorporated. These case statements decide when to switch from B lane to A lane and when to loop around the entire track. If all 2 objects are found then the robot stops, else if the robot loops around the track and doesn't sense all the objects, it then goes to B5 to look for the final object,
- **Default Case**; this case ends Line following and stops the robot.

4. Hardware

4.1 Parallax Propeller Activity Board

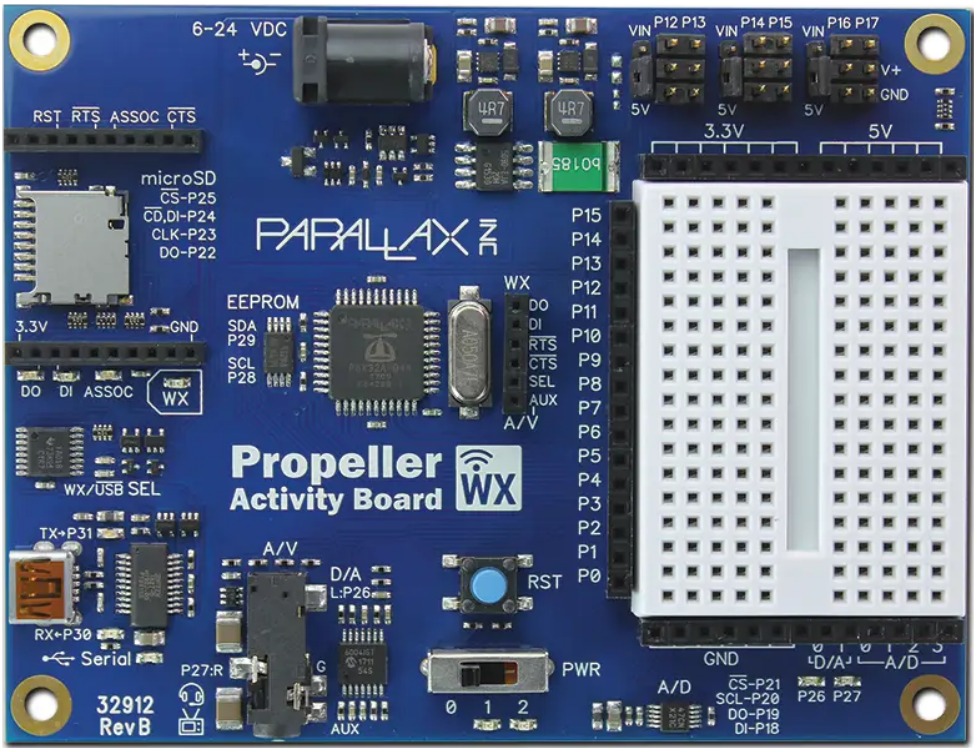

Figure 4.1: Parallax Propeller 2 Activity Board WX

The Propeller Activity Board WX features an 8-core P8X32A-Q44 microcontroller; it uses 6-24V external power supply. It has 3 different memory sections that have to be considered;

- Global RAM - 32KB
- Global ROM - 32KB
- Cog RAM - 2KB per Cog

All 32 Pins can be accessed and used as inputs or outputs by each Cog, set_direction() is the function that is used to set i/o, it takes a pin number as the first parameter and state(1 or 0) as the second parameter, set_output() sets the pin either high or low and follows the same syntax as set_direction(). The Propeller acts as a master to the arduino and receives the QTR sensor data processed by the arduino and uses that to make decisions.

4.2 Arduino Nano

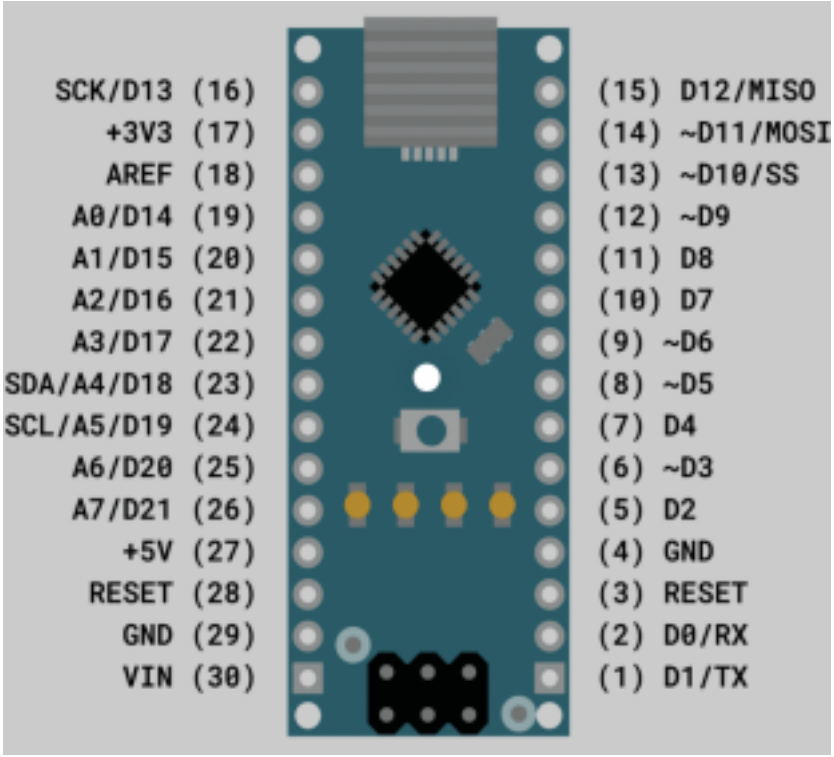

Figure 4.2: Arduino Nano Microcontroller

The open-source Arduino Nano uses a 6-20V unregulated power supply and is powered via the Mini-B USB connector. It has the same microcontroller as the Arduino Uno which is ATmega328. It has 3 different types of memory.

- Flash memory - 32kb.
- EEPROM - 1kb.
- SRAM - 2kb.

The Arduino acts as a slave to the Propeller and transfers the QTR Sensor data to the Propeller.

4.3 Ultrasonic Sensor

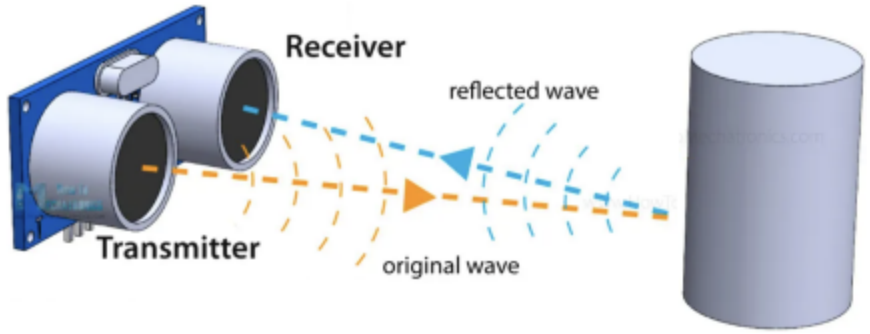

Figure 4.3: Ultrasonic waves transmitted and received by an Ultrasonic Sensor

The HC-SR04 Ultrasonic sensor is used to measure the distance by emitting an Ultrasound frequency of 40kHz. If any object is detected on the path, the Ultrasound waves bounce back to the receiver end as shown in Figure 3.3. The distance of the object is calculated by the halving time required for the Ultrasound waves to detect the object and bounce back to the receiver. It is calculated using the formula below;

$$Distance = \frac{Speed\ of\ sound\ in\ air \times Time\ from\ transmitter\ to\ the\ receiver}{2}$$

For example: Find the distance of the object if the time required to detect the object is 150 µs. Given: Speed of Sound in Air = 340 m/s = 0.034cm/µs

Time =1300 µs

Distance $= \frac{Speed \times Time}{2}$

$= \frac{0.034 \times 1300}{2}$

=22.1cm

General Pin Representation of Ultrasonic Sensor:

- Pin 1 - Vcc
- Pin 2 - Trigger
- Pin 3 - Echo
- Pin 4 - Ground

In this project, we're using 3 Ultrasonic sensors to detect the object and obstacles in 3 different directions(Right, Front and Left).

4.4 Pololu IR sensor Array

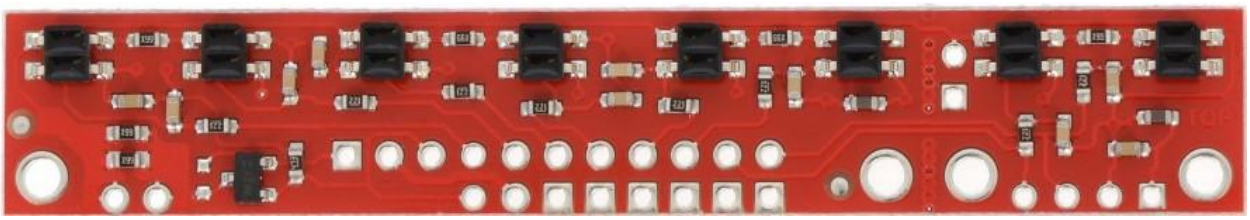

Figure 4.4: QTR-8RC Reflectance Sensor Array

With an in-built 8 IR LED, the IR sensor array is mostly utilized for sensing light and dark points on a plane, which just so happens to be fantastic for line-following robots. Pairing up with LEDs for low current consumption and MOSFET for power savings are some of the significant features of QTR-8RC. All 8 sensors have separate digital I/O measurable outputs. Apart from that, it doesn't require any analog to digital converter.

Specifications:

- Dimensions: 2.95" x 0.5" x 0.125" (74.93mm x 12.7mm x 3.175mm)
- Required operating Voltage : 3.3-5.0 V
- Required supply current: 100 mA
- Output format: 8 digital I/O-compatible signals
- Optimal sensing distance: 0.125" (3 mm)

In the current project, we're using the Pololu IR sensor array to detect the black line (tape) and follow the robot on the track given. The calibration of the IR sensor takes place which may require 7-10 seconds. Each IR returns a value of 0 for maximum reflectiveness to 1000 for minimum reflectiveness. Each IR LED is passed over the black tape for a split second and returned to the white paper background briefly, this is repeated as many times as possible for better calibration. The Pin representation of the Pololu sensor's connection to the Arduino Nano board was explained in the previous chapter.

5. Result and Conclusion

After the implementation of the line following robot with obstacle avoidance, 2 objects are kept at different locations (B5 and A4) and an obstacle was placed at the center lane (i2). In our demonstration video, the robot starts from a home location and continues down the center lane until it encounters an obstacle and then returns back on the center lane to i1. After that, it takes a turn to B1, and then scans for objects at the delivery location between B1 to B4, it then makes a turn on i4 and moves to A4, and scans from A4 to A1. Since the second object wasn't detected, the robot proceeds to move on to B5 and then end its journey.

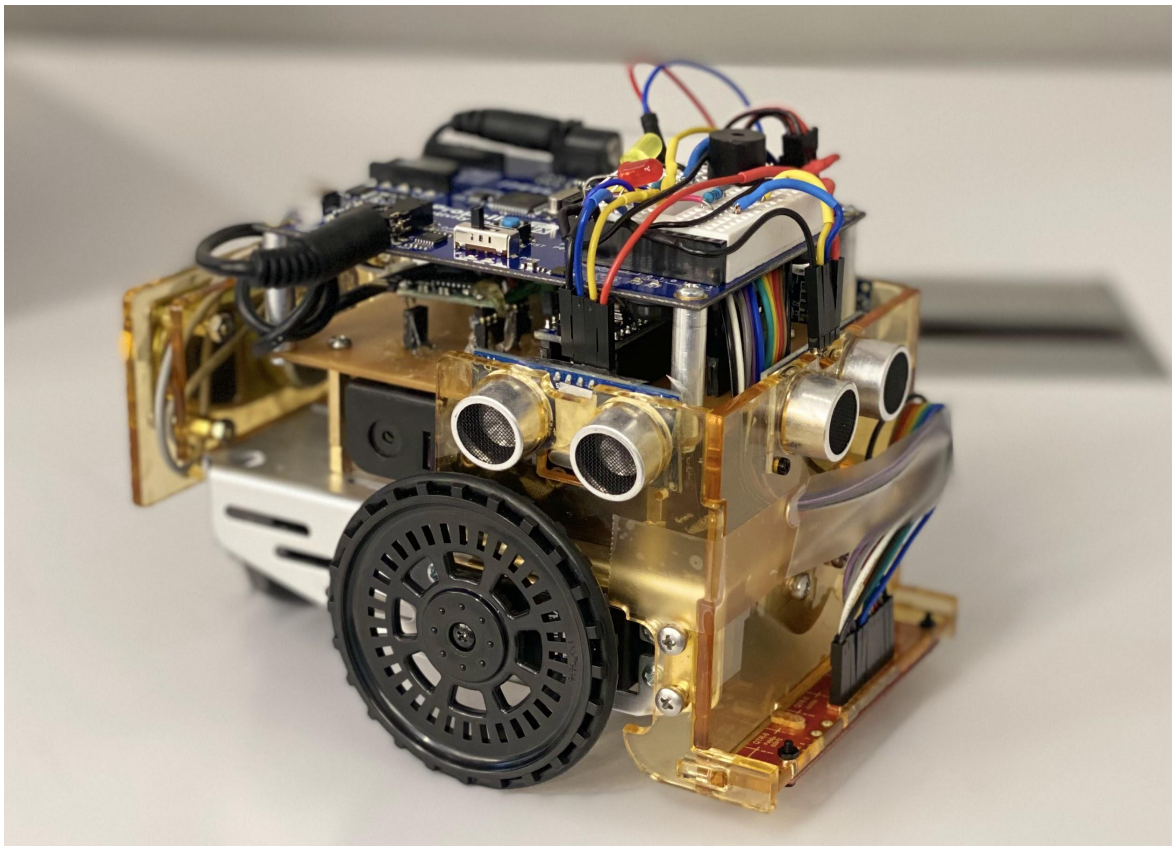

Figure 5.1: Picture of the Line Following Robot with Obstacle avoidance